The Open Source Audit

A Capstone Project for OSS NGMC Course

Chosen Software: Git-Version Control System

Student Name : Sneha

Reg No.: 24BCE11188

Slot : B22

Date Of Submission: 31st March 2026

Introduction

Linus Torvalds developed Git, a distributed version control system in 2005. This report is an organized audit of Git. Git is among the most influential pieces of open-source software ever created not just because of its functionality but also because of the reasons behind its development, the circumstances surrounding its creation and the principles it incorporates into every workflow it comes into contact with. There are four sections in this audit:

- Part A looks at the history of Git, the ethical issues raised by its creation and the philosophical applications of its GPL v2 license.
- The footprint of Git on a Linux system is described in Part B, including where it resides, how it is maintained, and what security features its packaging suggests.
- The ecosystem of platforms, communities, and tools made possible by Git is mapped out in Part C. Before reaching a deployment decision,
- Part D conducts an honest comparison between Git and its closest proprietary substitute, Microsoft's Team Foundation Version Control.

In addition to the written report, five shell scripts show off useful Linux skills that are directly related to the open-source ideology of automation and transparency. Every script has documentation that includes its code an explanation of the concepts it employs and execution instructions. Git was selected because it combines a number of significant open-source themes: it was developed by the man who created Linux it was born out of a licensing dispute and it is now the standard for collaborative software development globally.

Part A: Origin and Philosophy

A1. The Problem Git was Created to Solve

The Situation Before Git

Understanding what software development teams were using in the early 2000s and why those tools weren't working for them is a prerequisite to understanding Git. During that time, CVS (Concurrent Versions System)

and Subversion (SVN) its successor were the two most popular version control systems. Both used a centralized approach with each developer checking files into and out of a single server that contained the authoritative history of a project. No one could commit code if the server went down. Commits were slow if the network was slow. A project's entire history could be lost if the central repository was compromised.

These restrictions were inconvenient but acceptable for the majority of software teams. They were becoming unbearable for the Linux kernel project. A typical software project is not the Linux kernel. Thousands of contributors from dozens of nations submitted changes at the same time in 2005. The centralized tools that were available were not made for that kind of dispersed, high-velocity collaboration and merging contributions was difficult.

The Crisis of BitKeeper

BitKeeper, a proprietary distributed version control system developed by Larry McVoy's company BitMover, has been used by the Linux kernel project since 2002. There was always political tension when an open-source project used BitKeeper because many developers didn't want to use a proprietary tool to create a free operating system. Although the terms were always precarious, BitMover had provided free licenses to open-source projects which kept the arrangement bearable.

In April 2005, the agreement fell apart. In order to create a free client for the BitKeeper protocol, developer Andrew Tridgell, best known for creating Samba, started reverse-engineering the protocol. BitMover revoked the free licenses they had given Linux kernel developers because they saw this as a breach of their license terms. Suddenly, the kernel team and Linus Torvalds were without a version control system.

Git is built by Linus in two weeks

Linus Torvalds responded in his usual straightforward manner. He chose to write something new rather than use any of the existing systems (Monotone, Darcs, and Subversion existed but none met his needs for distributed design and performance). He started developing Git in April 2005. According to reports Torvalds's sardonic humor about the project

was reflected in the name which was self-deprecating—British slang for a disagreeable person. His demands were clear and strict.

Because Linus was benchmarking against CVS and required performance that was orders of magnitude better the system had to be quick. Every developer required a complete copy of the repository, independent of a central server so it had to be distributed. It had to ensure the history's integrity by identifying each commit using a cryptographic SHA-1 hash, which would prevent history from being secretly changed. Additionally, it had to facilitate non-linear development; thousands of branches and mergers had to be excellent operations rather than unpleasant outliers. Torvalds spent years considering what version control should be which is reflected in the speed at which Git was developed and the elegance of its fundamental design.

Reasons for Sharing It

From the start, Git was made available under the GNU General Public License version 2. This was not a coincidence. The most well-known GPL-licensed project in the world, Linux, was created by Linus Torvalds. He understood exactly what the GPL meant: anyone could take the code, alter it, and use it without restriction, but those changes could not be kept secret from the public. Git's release under the GPL was a conscious declaration that the tool for creating open-source software should be open-source in and of itself and that the BitKeeper scenario in which a community relied on a proprietary tool, should not be repeated.

A2. The License—What It Really Says

Git is available under the GNU General Public License version 2 (GPL v2). The GPL v2 is one of the most popular and important software licenses ever written. It shows that there is a carefully thought-out legal framework based on a simple but radical idea: once software freedoms are given they should never be taken away.

The Four Freedoms of Free Software

The Free Software Foundation says that a real free software license must guarantee four basic freedoms:

• Freedom 0: The right to use the program for any reason.

• Freedom 1: The right to learn how the program works and change it. You need access to the source code first.
• Freedom 2: The right to give copies to other people to help them.
•Freedom 3: The right to share copies of modified versions so that everyone in the community can benefit from the changes.
The GPL v2 license for Git gives all four freedoms without any conditions. Anyone can download, use, read, change, and share Git, even if they make changes to it. The most important condition is the copyleft clause: if you share a modified version, you must also share those changes under the GPL v2. You can't take Git's source code make it better and then sell a closed proprietary product. The freedom must move forward.

Can a business buy Git and not share any changes?

No, and this is the main difference between the GPL and licenses that are more flexible. If a company distributes a binary made from modified Git source code, they are legally required to make the source code available under the same GPL v2 terms. A business can create a commercial service that uses Git internally. GitHub does this and it doesn't have to make its platform open-source because providing a service is not the same as giving away software. But if that company sends out a product with a changed Git binary they must also make the changed source available to everyone who gets it.

GPL vs MIT — A Licensing Comparison

Dimension	GPL v2	MIT License
Copyleft	Strong: derivative works must stay GPL	None—derivatives can be private
Commercial Use	Allowed but the modified source must be shared	Allowed in full even if the source code is closed
Corporate Adoption Risk	Some businesses stay away from GPL code because of copyleft rules.	Very low—easy to add without having to do anything

Community Protection	Strong: a proprietary fork can't capture community labour	Weak—companies can take the work and shut it down.

If my main goal is to get as many people as possible to use my work and I'm okay with anyone using it without giving me credit I would choose the MIT license for my own projects. If I believe that the freedom to study and change software is important, I would choose the GPL. I want to make sure that no business can take improvements made by the community and keep them from the people who need them.

Free as in Freedom versus Free as in Free Beer

Richard Stallman is credited with making this distinction, which is crucial to comprehending the true meaning of open source. Free beer costs nothing. Unrestricted rights are referred to as freedom (libre in French). In one sense, software can be free, but not in another. Software that is given away for free but lacks source code is known as freeware; it is not libre. Gmail is free to use but it cannot be studied, altered, or shared. Git is both: it offers all four freedoms at no cost. Practically speaking the distinction matters because free software can change its terms or disappear at the company's discretion. Because the rights are incorporated into every distributed copy, free software cannot be taken away.

A3. The Ethics of Open Source

Should All Software Be Open Source?

When we ask if all software should be open source, we're really talking about who gets access to knowledge, and who holds power. Software isn't something you use up. If I read the source code for a program, I don't take anything away from you. But the code running through banks, hospitals, voting systems or social media isn't just lines of text—it shapes real decisions that affect all of us. Ordinary people and politicians have a real stake in knowing what's going on under the hood. If code stays locked away behind corporate walls, there's no real way for outsiders to see how these key decisions get made. You only get in if the owner lets you.

Security is another big piece. There's a classic cryptography rule: make the system public just keep the keys secret. Hiding how things work hoping nobody finds the weak spots, rarely stands up to scrutiny.But let's be real—open sourcing everything isn't a slam dunk. Building good software takes money, talent, and time. Companies need ways to pay for engineers, servers, and coffee. Selling licenses helps businesses stay afloat and keeps new features coming. Think of all the critical systems out there—medical devices, factory controls, enormous databases. Most of these exist because companies have figured out how to profit from keeping some of their work behind closed doors. If you force everything into the open, a lot of that might simply not get built or progress could stall due to lack of funding.

You've also got to consider quality. Open source is awesome when lots of developers care enough to get involved, review code, squash bugs, and pitch in ideas. That's why shared tools like web servers thrive; the community is massive.

In the end open source isn't a silver bullet. Sure it opens doors and helps make things safer and more transparent but it doesn't pay the bills or guarantee high-quality tools across the board. You have to weigh access and scrutiny against business realities and specialized needs.

The Ethics of Building on Free Labor

Big companies and open-source communities are locked in this ongoing debate about what's fair. Look at Linux: it's basically everywhere. About 90% of public cloud infrastructure runs on it. Amazon, Google, and Microsoft rake in huge sums—hundreds of billions of dollars—all fueled by Linux under the hood. Yet the Linux kernel itself? It's looked after by a pretty small group, a few paid by tech giants like Red Hat, Intel, and Google. But most of the core code, the stuff years in the making, came from people who never got a paycheck for their contributions.

Take Red Hat they carved out a multi-billion dollar empire by packaging, certifying, and supporting Linux and then IBM snatched them up for $34

billion. Is that ethical? Well you have to check what the GPL (the General Public License) actually asks from companies. It says you have to share any source code tweaks, and Red Hat's always played by that rule. But the GPL doesn't say you have to pay the original developers, chip in engineering hours, or invest back in the community. Red Hat does hire hundreds of kernel developers and feeds a lot back into the ecosystem. Honestly, it's all pretty gray. Corporations pull massive value from work the community provides for free, and apart from the license, the community doesn't have much power to force companies to return the favor. Efforts like the Linux Foundation or GitHub Sponsors try to build more sustainable support systems, but the friction—the gap between what the GPL demands legally and what developers feel is morally right—still hasn't gone away.

Standing on the Shoulders of Giants

People usually link this phrase to Isaac Newton, but in software, it's not just a nice turn of phrase — it's how the whole thing works. Every app I ever write will rely completely on what's come before. My code gets built with toolchains like GCC or LLVM (open source, of course), runs on something like Linux, probably uses Python or Node.js stores its data in PostgreSQL or SQLite and gets tracked in Git. All of it: open source.

Now, does open source boost real innovation or kill it? Honestly, it does both, and it depends on your point of view. If you're an indie developer or a tiny company, open source is the big unlock — you get to use tools for free that would've cost a fortune not that long ago. You can build something impressive from your bedroom. But if you're a company that sunk millions into private software open source just shrinks your profit margins and makes that big investment harder to justify. Still, if you look at the last three decades, it's pretty clear. Open source delivered more innovation and it did it faster than the old, closed-off, paywall-heavy world ever did.

<u>Part B — Linux Footprint</u>

Installation Method

Getting Git onto a Linux system is pretty painless, since every major Linux distro offers Git straight from their package manager. If you're running something Debian-based, like Ubuntu, you just open a terminal and type:

```
sudo apt update && sudo apt install git
```

If you're on Fedora or Red Hat, grab Git with:

```
sudo dnf install git        # Fedora / RHEL 8+
sudo yum install git        # CentOS 7 / RHEL 7
```

Arch fans, it's as simple as:

```
sudo pacman -S git
```

Now, if you need the absolute bleeding edge or you're hacking on Git itself, you'll want to build from source instead. You can grab the code from git-scm.com or GitHub (github.com/git/git). You'll need GCC, Make, and a handful of libraries like libcurl and zlib.

Key Directories

Once you've installed Git through a package manager, its files spread across the usual places:

- /usr/bin/git — That's the main program. Any time you type 'git' in the terminal, this is what runs. Other commands, like git-fetch or git-upload-pack, live here too.
- /usr/lib/git-core/ — All the behind-the-scenes helpers and scripts. A lot of Git's subcommands are actually separate programs tucked away in this directory.
- /usr/share/git-core/ — This holds things that aren't tied to any one machine like shell completion and repository templates.
- /usr/share/doc/git/ — Documentation, changelogs, and license info.

User and Group: When you run Git, it uses your own user account—there's no separate Git user or background process lurking on your system. That makes sense, since Git is basically handling your files from the command

line. It sticks to the usual Unix permissions, so Git only touches files you're allowed to access.

Starting with Git 2.35.2, there's a security feature you should know about: Git refuses to work in a repository owned by someone else unless you specifically mark that directory as "safe" in your Git configuration. This helps block attacks where someone tries to sneak a malicious .git/hooks/ folder into a shared spot.

Service Management:

Git isn't a service. It just runs your command and quits. But, if you want people to clone or push to a shared repo, you need something running on the server side. The simplest option is the Git daemon, which you manage with system like this:

- Start the git daemon service : sudo systemctl start git.socket
- Check its status : sudo systemctl status git.socket
- Stop the service : sudo systemctl stop git.socket

For private repos, most people use SSH access with Git

```
sudo systemctl status sshd      # See if SSH is running

sudo systemctl restart sshd     # Restart SSH if there's trouble
```

Update Model : Security patches and Git updates roll out through your distro's usual package manager. When someone finds a vulnerability in Git—like the path traversal bugs fixed in CVE-2022-24765—maintainers jump in, patch it, and push the update out the usual way:

```
sudo apt upgrade git        # For Debian / Ubuntu

sudo dnf upgrade git        # For Fedora / RHEL
```

This is a big reason why open-source rocks: everyone can see how security fixes are handled, from discovery to deployment. The CVE database keeps a public record of the vulnerabilities.

Shell Script Tasks

Script 1 — System Identity Report

```bash
#!/bin/bash
# Script 1: System Identity Report
# Author: Sneha| Roll: 24BCE11188
# Course: Open Source Software
# Description: Displays Linux system info and OS license details

# --- Variables ---
STUDENT_NAME="Sneha"
ROLL_NUMBER="24BCE11188"
SOFTWARE_CHOICE="Git"

# --- Gather system information using command substitution ---
KERNEL=$(uname -r)
DISTRO=$(grep PRETTY_NAME /etc/os-release 2>/dev/null | cut -d= -f2 | tr -d '"')
USER_NAME=$(whoami)
HOME_DIR=$(eval echo ~$USER_NAME)
UPTIME=$(uptime -p)
DATETIME=$(date '+%d %B %Y  %H:%M:%S')

# --- Display formatted output ---
echo "========================================"
echo "   Open Source Audit --- $STUDENT_NAME"
echo "   Roll Number      : $ROLL_NUMBER"
echo "   Software Choice  : $SOFTWARE_CHOICE"
echo "========================================"
echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "User         : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date & Time  : $DATETIME"
echo "OS License   : GNU General Public License v2 (GPL v2)"
echo "========================================"
```

This script generates a formatted welcome screen with information about the open-source license and system identity. An instantaneous snapshot of the Linux environment can be obtained by running it. Variables, echo, command substitution with $(), grep with cut and tr for text extraction, and date formatting are among the concepts that are illustrated.

Script 2 — FOSS Package Inspector

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Sneha| Roll: 24BCE11188
# Course: Open Source Software
# Description: Checks if Git is installed and prints package info + philosophy note

# --- Package to inspect ---
PACKAGE="git"

# --- Check installation: try RPM first, then DEB ---
echo "Checking if '$PACKAGE' is installed..."
echo "----------------------------------------"

if command -v rpm &>/dev/null && rpm -q $PACKAGE &>/dev/null; then
    # RPM-based system (Fedora, RHEL, CentOS)
    echo "$PACKAGE is installed (RPM-based system)"
    rpm -qi $PACKAGE | grep -E 'Version|License|Summary'

elif command -v dpkg &>/dev/null && dpkg -l $PACKAGE 2>/dev/null | grep -q '^ii'; then
    # DEB-based system (Ubuntu, Debian)
    echo "$PACKAGE is installed (DEB-based system)"
    dpkg -l $PACKAGE | grep '^ii'

elif command -v git &>/dev/null; then
    # Git Bash / Windows fallback
    echo "$PACKAGE is installed"
    echo "Version : $(git --version)"
    echo "License : GPL v2"
    echo "Summary : Fast, distributed version control system"

else
    echo "$PACKAGE is NOT installed."
    echo "Install it with: sudo apt install git"
fi

echo ""
echo "--- Philosophy Note ---"

# --- Case statement: prints a one-line philosophy note per package ---
case $PACKAGE in
    git)     echo "Git: born in two weeks to free Linux from proprietary version control." ;;
    httpd)   echo "Apache: the web server that built the open internet." ;;
    mysql)   echo "MySQL: open source at the heart of millions of web applications." ;;
    firefox) echo "Firefox: a nonprofit's fight for an open and independent web." ;;
    vlc)     echo "VLC: built by students who just wanted to stream video freely." ;;
    *)       echo "$PACKAGE: another essential block in the open-source foundation." ;;
esac
```

This script uses a case statement to print a philosophy note about the package after determining whether Git is installed and retrieving its version and package metadata. It supports both DEB-based (Debian, Ubuntu) and RPM-based (Red Hat, Fedora) systems. The following concepts are illustrated: pipe with grep for filtering, case statement with pattern matching.

Script 3 — Disk and Permission Auditor

```
# Course: Open Source Software
# Description: Loops through system directories and reports permissions + disk usage

# --- Array of important system directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================"
echo "       Directory Audit Report"
echo "========================================"

# --- For loop: iterate over each directory in the array ---
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # awk extracts: permission string, owner, group (fields 1, 3, 4)
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # du reports disk usage; cut extracts just the size value
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR"
        echo "  Permissions : $PERMS"
        echo "  Size        : $SIZE"
        echo "----------------------------------------"
    else
        echo "$DIR — does not exist on this system"
        echo "----------------------------------------"
    fi
done

# --- Bonus: Check Git configuration file ---
echo ""
echo "--- Git Configuration Check ---"
if [ -f "$HOME/.gitconfig" ]; then
    echo "Global Git config found:"
    ls -l "$HOME/.gitconfig" | awk '{print "  Permissions:", $1, "| Owner:", $3}'
    echo "  Contents preview:"
    cat "$HOME/.gitconfig"
else
    echo "No ~/.gitconfig found yet."
    echo "Run: git config --global user.name 'Your Name'"
    echo "Run: git config --global user.email 'your@email.com'"
fi
```

There are two primary functions of this script: audits critical system directories (size, ownership, and permissions) and it verifies the global configuration of Git.

Script 4 — Log File Analyzer

```
LOGFILE=$1                  # First argument: path to the log file
KEYWORD=${2:-"error"}       # Second argument: keyword to search (default: error)
COUNT=0                     # Counter variable, starts at zero

# --- Validate: check the user provided a file path ---
if [ -z "$LOGFILE" ]; then
    echo "Usage: ./script4_log_analyzer.sh <logfile> [keyword]"
    echo "Example: ./script4_log_analyzer.sh /var/log/syslog error"
    exit 1
fi

# --- Validate: check the file actually exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' not found."
    exit 1
fi

# --- Retry loop if file exists but is empty ---
ATTEMPTS=0
while [ ! -s "$LOGFILE" ] && [ $ATTEMPTS -lt 3 ]; do
    echo "Log file is empty. Waiting 5 seconds... (attempt $ATTEMPTS)"
    sleep 5
    ATTEMPTS=$((ATTEMPTS + 1))
done

echo "Analyzing: $LOGFILE"
echo "Keyword  : '$KEYWORD'"
echo "----------------------------------------"

# --- While read loop: go through file line by line ---
while IFS= read -r LINE; do
    # if-then: check if this line contains the keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # increment counter
    fi
done < "$LOGFILE"

# --- Print summary ---
echo "Total matches found : $COUNT"
echo ""
echo "Last 5 matching lines:"
echo "----------------------------------------"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

A log file path and an optional keyword are accepted as command-line arguments by this script. It counts the number of times a keyword appears in the file, reads it line by line, and prints a summary that includes the final five lines that match. The edge case of an empty log file is handled by a retry loop.

Script 5 — The Open Source Manifesto Generator

```
echo "========================================"
echo "   Open Source Manifesto Generator"
echo "========================================"
echo ""

# --- Interactive input using read -p ---
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# --- Generate metadata ---
DATE=$(date '+%d %B %Y')
AUTHOR=$(whoami)

# --- Output filename uses string concatenation with variable ---
OUTPUT="manifesto_${AUTHOR}.txt"

echo ""
echo "Generating your manifesto..."
echo ""

# --- Write to file: > creates, >> appends ---
echo "========================================"  > "$OUTPUT"
echo "       OPEN SOURCE MANIFESTO"            >> "$OUTPUT"
echo "========================================"  >> "$OUTPUT"
echo "Author : $AUTHOR"                         >> "$OUTPUT"
echo "Date   : $DATE"                           >> "$OUTPUT"
echo ""                                         >> "$OUTPUT"
echo "I believe in $FREEDOM."                   >> "$OUTPUT"
echo ""                                         >> "$OUTPUT"
echo "Every day I rely on $TOOL —"              >> "$OUTPUT"
echo "a tool built freely and shared without"   >> "$OUTPUT"
echo "restriction by people I will never meet." >> "$OUTPUT"
echo ""                                         >> "$OUTPUT"
echo "Standing on that foundation, I commit"    >> "$OUTPUT"
echo "to build $BUILD and release it freely"    >> "$OUTPUT"
echo "for anyone to use, study, modify,"        >> "$OUTPUT"
echo "and share. That is what it means"         >> "$OUTPUT"
echo "to work in the open."                     >> "$OUTPUT"
echo ""                                         >> "$OUTPUT"
echo "-- $AUTHOR | $DATE --"                    >> "$OUTPUT"
echo "========================================"  >> "$OUTPUT"

# --- Display the saved file ---
echo "Manifesto saved to: $OUTPUT"
echo ""
cat "$OUTPUT"
```

This imaginative script creates a customized philosophy statement and saves it to a text file named after the current user after asking the user three interactive questions about their open-source values.

Part C- The FOSS Ecosystem

Relationships

Because Git is written in C and has a purposefully small dependency footprint, it is portable across Linux distributions and processor architectures. The following are essential build-time and runtime

dependencies:
• When cloning or pushing over HTTPS, use OpenSSL or Gnu TLS for certificate validation and HTTPS transport security.
• libcurl: for proxy support and remote HTTP and HTTPS operations
• PCRE2—to support regular expressions in git grep --perl-regexp
• Perl—used by a number of auxiliary scripting tools and git send-email
• Python is utilized by some contrib scripts and git p4 (Perforce Bridge).
• get text: this feature enables the display of Git messages in the user's system locale for internationalization.

What Git Has Facilitated and Motivated

It is hard to overestimate Git's influence on the software ecosystem. It altered the structure of software collaboration rather than just replacing previous version control systems. Git enabled the following platforms, tools, and workflows:

• GitHub (2008), a web-based hosting platform for Git repositories that emerged as the world's de facto hub for open-source cooperation. It now houses more than 300 million repositories after being purchased by Microsoft for $7.5 billion in 2018.
• GitLab, a completely open-source substitute that provides issue tracking, container registries, integrated CI/CD pipelines and self-hosted repository management. The core of GitLab can be operated independently of any commercial service and is licensed by MIT.

• Gitea :a lightweight, community-managed Git server for groups seeking self-hosted, straightforward, resource-efficient infrastructure without the hassles of a full GitLab installation.

Git and the Web: A Link to the LAMP Stack

Although Git is not a runtime component of the LAMP stack (Linux, Apache, MySQL, PHP), it is essential to the development and deployment of contemporary LAMP applications. Git version controls the source code for MySQL schema migration scripts, PHP applications, and Apache configuration files. To push a particular code revision to a production server, deployment pipelines—whether they are straightforward shell

scripts or intricate CI/CD systems—usually pull from a Git repository. A Git commit identifier defines "what is deployed right now" in its entirety.

Community and Governance

Git's development feels old-school, but it's steady. Junio C Hamano has run the show since Linus Torvalds handed him the keys back in 2005, right after inventing Git. Hamano reviews code, sets up releases, and makes the final call on what gets in. This is the classic "benevolent dictator" setup — not unlike how Python was run before it moved to a more formal model.All the action happens on the git mailing list. There's no web forum; you have to send formatted patches via 'git send-email,' get feedback, revise, and eventually merge. It's a demanding workflow — let's be real, GitHub pull requests are easier — but it gets results: the codebase stays reliable and high-quality.

Every two to three months, a new release drops. Unlike Linux, which has the Linux Foundation behind it, Git doesn't rely on a big umbrella organization. Most of the work is done by volunteers and by contributors whose employers want to keep Git in good shape. Google's Summer of Code has helped out too, funding quite a few notable improvements over the years.

Part D – Open Source vs Propreitary

Dimension	Git(Open Source,GPL v2)	Perforce Helix Core(Proprieatry)
Cost	Git is free. You only start paying when you need hosted services like GitHub Teams or a premium GitLab setup. The core tool? Zero dollars	Perforce isn't even in the same league price-wise. If you want an enterprise license, think in terms of hundreds of thousands a year. Not exactly pocket change.
Code Availability	You can read every single line of Git's code if you want.	You have no idea what's under the

		hood—no way to audit it yourself
Support	There's no shortage of Git support—forums, docs, commercial services through folks like GitHub or GitLab, or third parties.	With Perforce, you get promises and SLAs, but only from Perforce.
Governance	Git is community-driven and open	Perforce, though, is totally vendor-controlled.
Overall Verdict	Most teams should just use Git—it's fast, free, and driven by the broader community.	Perforce makes a lot of sense. Otherwise, Git wins.

Conclusion And Recommendations

After digging into Git's history, how it works, its license, the community around it, and where it stands among competitors, here's my take: Git is the tool you want for version control, no matter who you are—whether you're just getting started with your first project, or you're managing a team of hundreds at a massive company. It doesn't cost a dime it handles projects of any size, and it's designed so no company can swoop in and lock you out or change the rules on a whim. And should you consider contributing to Git itself? Absolutely. Despite how widely it's used, Git's development is still very much in the community's hands. Even the tiniest fix—whether it's a better sentence in the docs, a speedup, or a more robust way of handling weird edge cases—ends up making life better for pretty much every software developer out there. That's time well spent, and it's exactly how open-source keeps getting stronger.